Pre-lab 5 and Lab5 - Introduction to SDN and OpenFlow Building a Firewall

Total Points: 100 (5 points for progress check in lab)

Complete before you come to lab

Before getting started: Read the sections below about Software Defined Networking and OpenFlow. Then answer the following questions:

Software-Defined Networking & OpenFlow:

Software-Defined Networking (SDN) is a networking paradigm in which the data and the control planes are decoupled from one another. One can think of the control plane as being the network's "brain," i.e., it is responsible for making all decisions (for example, **how** to forward data), while the data plane is what actually moves the data. In traditional networks, both the control and data planes are tightly integrated and implemented in the forwarding devices (such as the IP Routers).

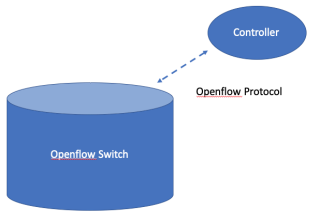
The SDN control plane is implemented by the "controller" and the data plane by the "switches." The controller acts as the "brain" of the network and sends commands (AKA "rules") to the switches on how to handle traffic (packets). The OpenFlow protocol has emerged as the de facto SDN standard that specifies how the controller and the switches communicate with one another, as well as how the controllers install the rules onto the switches.

Mininet and its Controller:

In Lab 1, we experimented with Mininet using its internal controller (and as such did not need to worry about OpenFlow). In this lab (and the next one), we will use our own controller (AKA a remote controller) to send commands to the switches. In the SDN network, the controller can be the default one in Mininet or any one of many different **remote controllers** that are available, such as POX, Ryu, *et. al.* The remote controller we are using is the POX controller and it is written in Python. The commands sent by the POX controller to the switches will be written by you!

OpenFlow 1.3 Overview:

OpenFlow 1.3 is the version of the OpenFlow protocol supported within the Mininet environment. The OpenFlow protocol is a standard communication protocol to enable and manage the communication between the controller and switches. As you would expect with any standard protocol, OpenFlow operates irrespective of the vendor of the controller or switches and also different hardware.



The following diagram explains the matching operation of OpenFlow switches.

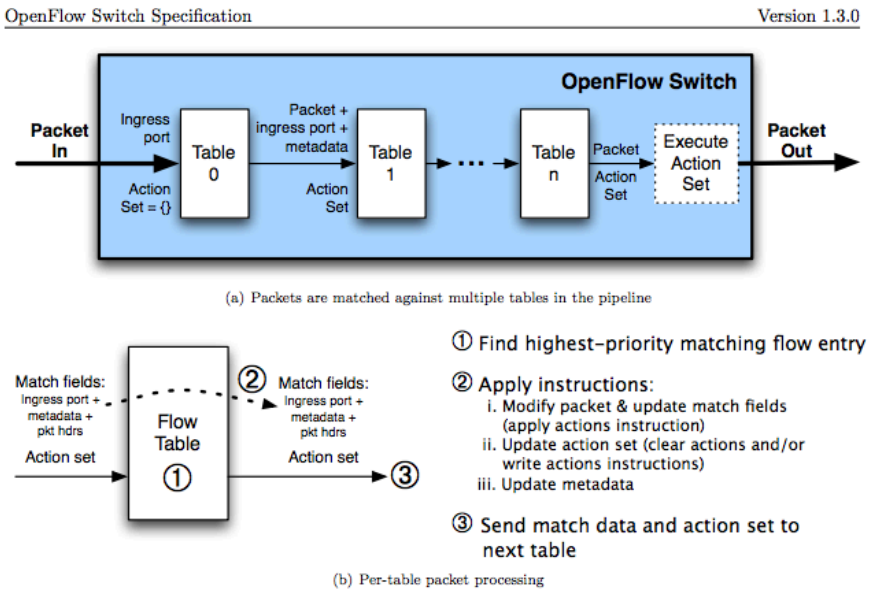


Figure 2: Packet flow through the processing pipeline

Note that when the packet arrives at an OpenFlow switch, the switch will reference a table containing “rules” and “actions”. The entries in the flow table contain the following fields:

OpenFlow Switch Specification Version 1.3.0

Match Fields	Priority	Counters	Instructions	Timeouts	Cookie

Table 1: Main components of a flow entry in a flow table.

The flowchart below shows the flow of execution:

- If an `ofp_packet_in` does not match any of the flow entries in the flow table and the flow table does not have a “table-miss” flow entry, the packet will be dropped.
- If the packet matches the “table-miss” flow entry, it will be forwarded to the controller.
- If there is a match-entry in the flow table for the packet, the switch will execute the action stored in the instruction field of the corresponding flow table.

5.3 Matching

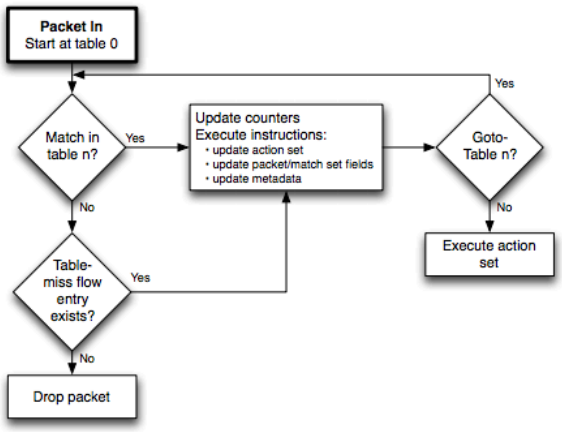


Figure 3: Flowchart detailing packet flow through an OpenFlow switch.

All of the figures and information in this section are from the [OpenFlow 1.3 specification](), which you can reference for additional information.

Question 1 [5 pts]: Introductory Questions:

1. **Use your own words to answer the following questions** and for each question, provide your reference from either the References section listed below within this document:

a. What is Software Defined Networking?

Software-defined networking (SDN) is a network management approach that uses software to control and direct network traffic.

SDN separates the network's control plane from the data plane, with the control plane operated by software and the data plane operated by hardware. This allows network administrators to manage data traffic programmatically, rather than relying on traditional hardware devices like routers and switches

https://www.cisco.com/c/en/us/solutions/software-defined-networking/overview.html
https://www.ibm.com/topics/sdn#:~:text=SDN%20solutions%20come%20with%20significant,network%20to%20ensure%20business%20continuity.&text=Service%20providers%20and%20organizations%20alike,private%20networks%2C%20MPLS%20and%20others.

b. What is OpenFlow?

OpenFlow is an open-source communication protocol that allows for centralized management of a network by providing a standardized way for

controllers to communicate with network devices.

https://www.techtarget.com/whatis/definition/OpenFlow

c. What is Mininet?

Mininet is a free, open source software tool that allows users to create virtual networks on a computer or virtual machine.

http://mininet.org/overview/#:~:text=Mininet%20is%20a%20network%20emulator,for%20network%20creation%20and%20experimentation

d. What is the purpose of a Controller in Mininet?

The controller acts as a central point of management and configuration of the openFlow switches within a simulated network. It dictates how packets flow through the network by defining rules on the switches.

http://mininet.org/overview/#:~:text=Mininet%20can%20create%20kernel%20or,virtual%20ethernet%20(%20veth%20)%20pairs.

e. What is the POX Controller? How is it different from the Mininet default controller?

The POX Controller is an open-source SDN controller written in Python, commonly used in SDN research and development for managing OpenFlow networks. Unlike the Mininet default controller, which is simple and primarily for basic operations, POX offers more advanced functionality and customization. It provides a versatile platform for building SDN applications, implementing network protocols, and experimenting with more complex network control logic, making it a preferred tool for many SDN developers.

https://courses.cs.washington.edu/courses/cse461/21wi/section-slides/section3.pdf

Lab 5 - Introduction to SDN and OpenFlow
Building a Firewall

References:

- **Firewalls**
  - Computer Networking: Section 4.3 “Inspecting Datagrams: Firewalls and Intrusion Detection Systems”
- **Mininet:**
  - On the Mininet site, the API Reference will be an excellent resource for figuring out how to run pings or open the command prompt in between the `net.start()` and `net.stop()` lines.
  - A Mininet Walkthrough can be found on this page
- **POX Controller/ OpenFlow:**
  - https://noxrepo.github.io/pox-doc/html/#
  - POX WIKI
  - Inside your VM, the `pox/pox/forwarding/l2_learning.py` example file

    Sending OpenFlow messages with POX
  - **Link to OpenFlow Tutorial**: here
  - OpenFlow 1.3 specification
- **IP Header**
  - Protocol Numbers in IP Header (Protocol Field)

Background and Goals of this lab:

This lab builds on the knowledge acquired in previous labs where you were first introduced to the Mininet environment and some of Mininet’s basic functionality. Now we will take that one step further by introducing you to Software-Defined Networking (SDN) and the OpenFlow protocol. (See tutorial in References.) After familiarizing yourself with these topics you will build your own firewall.

This lab assumes you will spend significant time reading the documentation and learning about Mininet, OpenFlow and the POX controller. ***It’s important to read the below write up before starting the lab and look at the above references.***

Lab Environment: All of the exercises in this lab have been run in the lab. We cannot provide support for other environments – some things might work and others not etc. Scripts must be written in Python 3.

Submission:

1. **lab5.pdf**: Answers and screenshots for all the questions. Submitted on Gradescope.
2. <cruzID>.**`lab5controller.py`**: Your firewall code submitted on Canvas.

Code provided for you:

- The Mininet configuration, **`lab5.py`**, to set up the network in Topology 1 which uses a remote controller
- A skeleton POX controller, **`lab5controller.py`**

Screenshot Check-off: (just for you to keep track of your progress)

3 a
5 a,b
8 a, b
9 c
10

Question 2 [5 pts]: Default Topologies

a. What command invokes the controller to create a tree topology where the "root" switch is connected to 3 other switches, and each of these switches is connected to 3 unique hosts?

   $ sudo mn --topo tree,depth=2,fanout=3 --controller remote

b. Use a drawing tool to draw an image of this topology, labeling the switches and hosts with the IP addresses assigned to their interfaces.

```
                Switch 1 (Root)
                  /    |    \
                 /     |     \
      Switch 2     Switch 3     Switch 4
      /  |  \        /  |  \        /  |  \
   h1   h2  h3    h4  h5  h6    h7  h8   h9
```

c. How many unique paths from a given host to any other host are there in this topology? What is the maximum number of hops between any two nodes?
   Because this is a tree topology, there is only one unique path between any two hosts. (No redundant paths).

   Maximum number of hops between any two nodes is 4 hops (host → switch → root → switch → host)

Understanding and Building Your Firewall:

Question 3 [5 pts]: Controllers in Mininet

In the Mininet environment different controllers can be activated:

- "sudo mn" invokes Mininet along with its internal controller
- "sudo mn --controller=remote" invokes a remote controller

a. **Default Controller:** Start Mininet with its internal controller. Once Mininet has finished setting up, run the command "dump"

- What is the IP address and Port number of the controller being used by Mininet? Include a **screenshot** of the output of the command and **circle in red** the IP address and Port number the controller is using.

```
mininet@mininet-vm:~$ sudo mn --controller=remote
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Unable to contact the remote controller at 127.0.0.1:6633
Setting remote controller to 127.0.0.1:6653
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=18471>
<Host h2: h2-eth0:10.0.0.2 pid=18475>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=18480>
<RemoteController c0: 127.0.0.1:6653 pid=18463>
mininet>
```

IP address: 127.0.0.1
Port number: 6653

b. **Remote Controller**: What command would you use to invoke a remote controller with IP Address 127.0.0.1 and port 9000?

$ sudo mn --controller=remote,ip=127.0.0.1,port=9000

Question 4 [5 pts]: Let's choose the POX Controller !

a. Why are we using the POX controller (instead of default) to implement the firewall?

   Pox controller allows for custom logic to be implemented, such as specific firewall rules. (Mininet only provides basic, limited functionality that does not allow for complex packet processing or security policies.) Also, POX supports the OpenFlow protocol, which is essential for SDN environments where custom traffic control and behavior are required.

b. Suppose you were going to use your topology in Question 2 with the POX controller. Add the POX controller to your drawing in Question 2 using arrows to show the communication between the devices and the controller. Label the hosts and switches as they would be referenced in Mininet and include the IP address and port number on the controller that you will soon be using in Topology 1 (you can find this in the provided topology file). Include your updated drawing here.

   The default port was 6653, but the given port in the topology file was 6633.

   Note: **All the lines are arrows that go both ways!!**

h1 = 10.0.0.1 h4 = 10.0.1.1 h7 = 10.0.2.1
h2 = 10.0.0.2 h5 = 10.0.1.2 h8 = 10.0.2.2
h3 = 10.0.0.3 h6 = 10.0.1.3 h9 = 10.0.2.3

Question 5 [5 pts] OpenFlow Messages

Work through the example here to see the OpenFlow messages generated by running Ping on a simple network.

Start Wireshark and filter to capture the OpenFlow messages. Start Mininet by running the simple topology in Question 2.

a. Show a **screenshot** of the terminal output when Ping is run between h3 and h9 (with the count adjusted to 2). Circle the communication between h3 and h9 in the screenshot.

```
mininet@mininet-vm:~$ sudo mn --topo tree,depth=2,fanout=3
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6 h7 h8 h9
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(s1, s2) (s1, s3) (s1, s4) (s2, h1) (s2, h2) (s2, h3) (s3, h4) (s3, h5) (s3, h6) (s4, h7) (s4, h8) (s4, h9)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet> h3 ping -c 2 h9
PING 10.0.0.9 (10.0.0.9) 56(84) bytes of data.
64 bytes from 10.0.0.9: icmp_seq=1 ttl=64 time=6.74 ms
64 bytes from 10.0.0.9: icmp_seq=2 ttl=64 time=1.09 ms

--- 10.0.0.9 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.092/3.917/6.743/2.825 ms
mininet>
```

b. Show a **screenshot** of the Wireshark output to show any of the following packets: `Packet-In`, `Packet-Out`, `Flow-Mod` and `Flow-Expired`

No.	Time	Source	Destination	Protocol	Length	Info
150	14.353663293	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
152	14.353672138	be:50:68:50:c1:77	16:56:a1:b5:37:28	OpenFl…	132	Type: OFPT_PACKET_OUT
154	14.357980180	be:50:68:50:c1:77	16:56:a1:b5:37:28	OpenFl…	126	Type: OFPT_PACKET_IN
155	14.358001829	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
157	14.358009926	be:50:68:50:c1:77	16:56:a1:b5:37:28	OpenFl…	132	Type: OFPT_PACKET_OUT
159	14.358174644	be:50:68:50:c1:77	16:56:a1:b5:37:28	OpenFl…	126	Type: OFPT_PACKET_IN
160	14.358194200	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
162	14.358201124	be:50:68:50:c1:77	16:56:a1:b5:37:28	OpenFl…	132	Type: OFPT_PACKET_OUT
164	14.358450051	10.0.0.3	10.0.0.9	OpenFl…	182	Type: OFPT_PACKET_IN
165	14.358469994	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
167	14.358477096	10.0.0.3	10.0.0.9	OpenFl…	188	Type: OFPT_PACKET_OUT
169	14.358689771	10.0.0.3	10.0.0.9	OpenFl…	182	Type: OFPT_PACKET_IN
170	14.358708697	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
172	14.358715439	10.0.0.3	10.0.0.9	OpenFl…	188	Type: OFPT_PACKET_OUT
174	14.358813096	10.0.0.3	10.0.0.9	OpenFl…	182	Type: OFPT_PACKET_IN
175	14.358830451	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
177	14.358837182	10.0.0.3	10.0.0.9	OpenFl…	188	Type: OFPT_PACKET_OUT
179	14.359033645	10.0.0.9	10.0.0.3	OpenFl…	182	Type: OFPT_PACKET_IN
180	14.359051253	127.0.0.1	127.0.0.1	OpenFl…	146	Type: OFPT_FLOW_MOD
182	14.359057805	10.0.0.9	10.0.0.3	OpenFl…	188	Type: OFPT_PACKET_OUT
184	14.359237626	10.0.0.9	10.0.0.3	OpenFl…	182	Type: OFPT_PACKET_IN

c. Explain the purpose of each OpenFlow message that you found and circled in (b).

Packet-In

- The switch sends a Packet-In message to the controller when it receives a packet that doesn't match any existing flow table entries. This informs the controller about the new traffic and requests instructions on how to handle it.

Packet out

- The controller uses Packet-Out to send a packet out through a switch's specific port. This action can be used to forward a packet that triggered a Packet-In or inject new packets into the network.

Flow-mod

- Flow-Mod messages are used by the controller to add, modify, or delete flow entries in the switch's flow table. These entries dictate how the switch should handle future packets that match specific criteria.

Flow-expired

- Indicates that a flow entry has been removed from the switch's flow table due to an idle_timeout or hard_timeout expiring. (No flow-expired messages were seen in my wireshark screenshot though)

Requirements, Guidance and Getting Started with the Firewall and POX:

A firewall is a wall you place in buildings to stop a fire from spreading. In the case of networking, it is the act of providing security by not letting specified traffic pass through the firewall into the internal network. This feature is good for minimizing attack vectors and limiting the network "surface" exposed to attackers.

Now we are going to learn a bit more about firewalls and how you can create your own firewall using OpenFlow-enabled switches. In this process, you will customize the remote controller to implement a given set of rules.

How to Get Started:

- Read documentation resources as needed
- Study the file `pox/pox/forwarding/l2_learning.py`

The firewall you build will be used by the small network in Topology 1. To build the firewall, modify the skeleton code in `lab5controller.py`. As you develop your firewall, please observe the naming convention outlined in **Table 1**. Your firewall will implement a set of rules which are outlined for you as **High level rules.** **Table 2** enumerates the Basic Rules that must be implemented**.** The topology itself has already been created for you in `lab5.py` and you do not need to do any modifications to this file.

Before you begin coding, there are several important aspects of implementing the firewall to understand: the firewall rules; how the rules are "installed" in the switches; how the switches will forward packets that they receive, and the underlying key function calls required to implement the firewall. Q 6, 7, and 8 get you into position to begin coding.

- **Note: Short-term memory loss! Rule installation**: When you create a rule in the POX controller, instruct POX to "install" the rule so that the switch "remembers" what to do**.** Hint: To do this, look up `ofp_flow_mod` `idle_timeout` and `hard_timeout` in the POX WIKI**.**

- **Forwarding packets**: In this assignment, when the action received from the controller is to forward a packet, you will implement the switches to forward packets through **flooding**. Your TA will discuss this with you in lab.

Question 6 [20 pts] Basic Implementation – working with the controller:

a. What are the lines of code to "install" the rules in the switch for an idle_timeout of 45 seconds and a hard_timeout of 10 minutes?

```
msg = of.ofp_flow_mod()
msg.match = of.ofp_match.from_packet(packet)
msg.idle_timeout = 45  # Set idle timeout to 45 seconds
msg.hard_timeout = 600  # Set hard timeout to 600 seconds
msg.actions.append(of.ofp_action_output(port=out_port))
self.connection.send(msg)
```

b. Think about accept() function and the timeout statements (the line of code in your accept() function). Suppose **`ping`** is run with a count >1 within your topology. As **`ping`** runs, what would happen in the switch if you forgot to include the timeout statements within your accept function? Explain in words and/or with a drawing if that helps.

Without the timeout statements, the flow entry would stay in the switch indefinitely until the switch is rebooted or the rule is manually removed. This could lead to resource exhaustion, as the switch's flow table could fill up with stale entries. When running ping with a count greater than 1, the switch would remember and apply the rule without checking with the controller again. Without a timeout, stale flow entries could prevent new traffic from being efficiently processed.

c. In your own words, what is packet flooding?

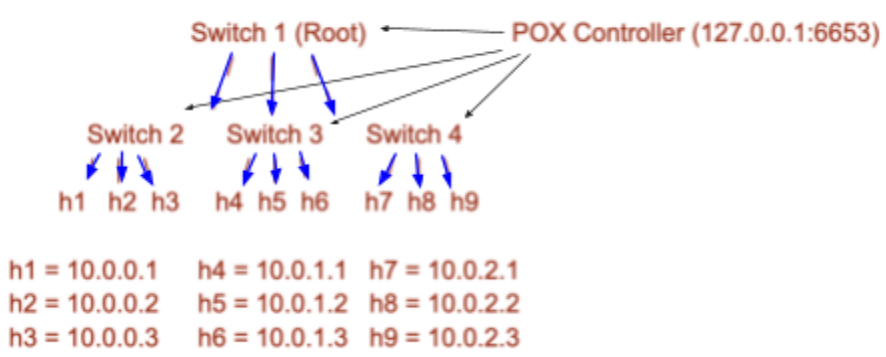
Packet flooding is a process where a network switch forwards incoming packets out of all ports (except the one it was received on) when it doesn't know the destination port. Flooding helps ensure the packet reaches its intended destination by sending it across all connected segments.

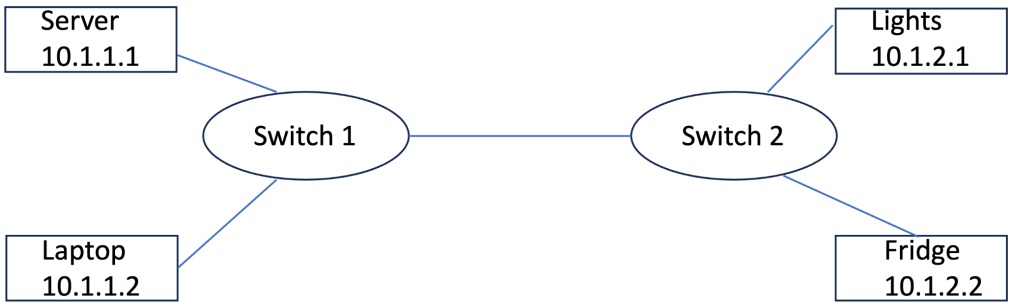
d. In your own words, what is port forwarding?

Port forwarding is a network technique where a router/switch forwards incoming traffic from a specific port to a designated device or application within a local network. This method is often used for allowing external devices to access services on a private network, such as hosting a website or accessing a private server.

e. Go back to your drawing in Question 4 and show a packet entering on one port and then show packet flooding. The solution here should include all requirements from Q2 and Q6

Blue arrows = flooding & black arrows: packet-in





Topology 1

Requirement: In your code use the naming convention below:

Device	**Required Naming Convention**
Laptop	'laptop'
Server	'server'
Fridge	'fridge'
Lights	'lights'
Switch 1	's1'
Switch 2	's2'

Table 1: Naming convention

High-level rules:

1. **General Connectivity:** Allow all ARP and ICMP traffic across the network to facilitate general network connectivity.
2. **Web Traffic:** Allow all TCP traffic between the laptop and the server.
3. **Access to IoT devices:** Allow all TCP traffic between the laptop and the lights. Allow all UDP traffic between the laptop and the fridge. Note: server does not access IoT devices
4. **Laptop/Server General Management:** Allow all UDP traffic between the laptop and the server.
5. **Default Deny**: Block all traffic that does not match the above criteria.

Rule #	Src Host	Src IP	Dst Host	Dst IP	Protocol	Action
1		any		any	ARP	accept
2		any		any	ICMP	accept
3	laptop		server		TCP	accept
4	server		laptop		TCP	accept

Rule #	Src Host	Src IP	Dst Host	Dst IP	Protocol	Action
5	laptop		lights		TCP	accept
6	lights		laptop		TCP	accept
7	laptop		fridge		UDP	accept
8	laptop		server		UDP	accept
9		any		any	ANY	drop

Table 2 - Basic Firewall Rules

Question 7 [15 pts] Basic Firewall Rules - Test your Understanding

a. Fill in the table to indicate which Basic Firewall Rule(s) in Table 2 support the implementation of the corresponding High-Level Rule:

High-Level Rule	**Rule(s) # in Table 2**
Rule #2 (Web Traffic)	3, 4
Rule #3 (IoT access)	5, 6, 7
Rule #4 (Laptop/Server General Management)	8

b. What activities are allowed for users Laptop/Server LAN?

ARP, ICMP, TCP, UDP

c. What activities are allowed between the IoT devices?

No default rules allow them. Hence, IoT devices cannot communicate directly with each other unless covered by rule #1.

d. Referring to Topology 1 and Table 2: **Suppose a TCP packet is sent from the server to lights.** Will this packet be allowed? State the rule number(s) that allows or disallows the transfer and explain the rule(s).

No, it will be disallowed because a TCP packet sent from the server to the lights does not match any of these specific rules in table 2. Server can't access any of the IoT devices, so rule 9 drops the packets.

e. If you were to run `iperf` between *the server* and *the lights*, do you expect it to work? Explain your answer based on the rules in Table 2.

While there is a link, the server doesn't have any connection to lights. Whether it's UDP or TCP there is no rule that allows the traffic from server to lights.

f. If you were to run '`traceroute -I`' the server and the lights, do you expect it to work? Explain your answer based on the rules in Table 2.

It will work since the -I uses ICMP packets and rule #2 allows for all ICMP traffic.

g. Write pseudocode for your Firewall using the rules in Table 2**.** Use the host names instead of the IP addresses in your pseudocode. e.g., use `lights` rather than `10.1.2.1`, according to the standard names in **Table 1**. Check `pox/pox/forwarding/l2_learning.py` to identify the key functions needed in your firewall. **Note: Rule #1 and #2 MUST be implemented FIRST**

```
if packet is arp:
    accept()
else if packet is icmp:
    accept()
else if packet is udp:
    # two way communication!!
    # src and dst are the srcip and dscip from the ip_header
   if(src == laptop and dst == server):
        accept()
    elif(src == server and dst == laptop):
        accept()
    elif(src == laptop and dst == fridge):
        accept()
    elif(src == fridge and dst == laptop):
        accept()
    else:
        drop()
else if packet is tcp:
    if(src == laptop and dst == server):
        accept()
    elif(src == server and dst == laptop):
        accept()
    elif(src == lights and dst == laptop):
        accept()
    elif(src == laptop and dst == lights):
        accept()
    else:
        drop()
else:
    drop()
```

How to run the POX Controller:

Your code: Save a backup of code frequently! Remember to follow the naming convention in Table 1. When your code is complete, put your firewall code into the **`~/pox/pox/misc`** directory.

To run this assignment, both files must run at the same time as follows:

In 2 different terminal windows:

- Launch the controller file:
  `sudo ~/pox/pox.py log.level --packet=WARN misc.lab5controller`
- Put the mininet topology file in ~ and then run it:
  `sudo python ~/lab5.py`

(It should make sense to you why the order matters.)

Question 8 [15 pts] Code complete - Testing the Basic Firewall

a. **pingall**:
   - Run the pingall command. Is the output correct? Which of the firewall rules contributed to the result? Explain your answer(s).
   - Include a **screenshot of the results.**

```
mininet> pingall
*** Ping: testing ping reachability
Fridge -> Lights laptop server
Lights -> Fridge laptop server
laptop -> Fridge Lights server
server -> Fridge Lights laptop
*** Results: 0% dropped (12/12 received)
mininet>
```

The output shows that the devices were able to ping each other successfully. This suggests that the current firewall rules are allowing ICMP traffic across the network for all hosts. This is contributed by rule#2 which is the general ICMP rule that accepts ICMP traffic between any source and destination.

b. **iperf:**
   - Let's test the TCP connection of your basic firewall.. For this, use the iperf command (see man page) to simulate TCP traffic and fill out Table 3 **Please make sure to include screenshots for #3 and #4.** (the first one has been completed - make sure your output agrees!).

#	Link	iperf command	Pass or Fail?
1	*Server - Lights*	`iperf server lights`	FAIL (if none-responsive use "Ctrl + C")

2	*Server - Fridge*	iperf server fridge	FAIL
3	*Laptop - Server*	iperf laptop server	Pass
4	*Laptop - Lights*	iperf laptop lights	Pass
5	*Lights - Fridge*	iperf lights fridge	Fail
6	*Fridge - Server*	iperf fridge server	Fail
7	*Fridge - Laptop*	iperf fridge laptop	Fail

Table 3 - Iperf testing table

#3:

```
mininet> iperf laptop server
*** Iperf: testing TCP bandwidth between laptop and server
*** Results: ['24.5 Gbits/sec', '24.5 Gbits/sec']
```

#4:

```
mininet> iperf laptop Lights
*** Iperf: testing TCP bandwidth between laptop and Lights
*** Results: ['31.7 Gbits/sec', '31.8 Gbits/sec']
```

I know the naming convention was supposed to be “lights,” but the lab5.py was using “Lights” instead. I didn’t change anything to lab5.py and just used it as it was.

c. Traceroute

The TA told me that UDP is always assumed to be two way

Protocol	Link	Traceroute Command	Pass or Fail?
ICMP	*Server - Lights*	server traceroute -I lights	Pass
UDP	*Laptop - Server*	laptop traceroute server	Pass
UDP	*Server - Laptop*	server traceroute laptop	Pass
UDP	*Laptop - Fridge*	laptop traceroute fridge	Pass

<u>Question 9 [10 pts]</u> Adding a new Firewall Rule – Ping Flood:

Ping flood is a common Denial of Service (DoS) attack in which an attacker takes down a DNS server by overwhelming it with Ping requests.

a. To protect your server in Topology1 from the Ping flood attack, what is the rule you would add to Table 2? Add this new Firewall rule for the Server in Table 4 below. Show the rule in the format of the rows in Table 2 and indicate the position where you will place this rule.

The new rule that needs to be added must block all ICMP traffic that has the server as the destination.

Src Host	Src IP	Dst Host	Dst IP	Protocol	Action
any	any	server	10.1.1.1	ICMP	drop
any	any	Anything but the server	Anything but the 10.1.1.1	ICMP	accept

Table 4 - Firewall table

b. Command: What command(s) will you use to test the firewall logic?

$ laptop ping -c 20 server

c. Run your test(s) and include a **screenshot.** Explain how your rule(s) are working.

```
mininet> laptop ping -c 20 server
PING 10.1.1.1 (10.1.1.1) 56(84) bytes of data.

--- 10.1.1.1 ping statistics ---
20 packets transmitted, 0 received, 100% packet loss, time 19453ms
```

When a ping is sent to the server, none of the traffic gets through, demonstrating that all ICMP packets with destination server is being blocked as we intended.

Question 10 [10 pts] Final Completed Firewall: Let's check out your full Firewall!

Perform the following commands in mininet, that checks if your rule to save your Server from DoS attacks work.There are also commands to check your UDP rules. Add if your commands passed or failed. **Please make sure to include screenshot for #1 and #4.**

#	Command	Pass or Fail?
1	`server traceroute -I laptop`	Fail
2	`Fridge ping server`	Fail
3	`Fridge ping laptop`	Pass
4	`iperfudp bw=20 laptop Fridge`	Pass
5	`server traceroute -U Fridge`	Fail

 # 1

```
mininet> server traceroute -I laptop
traceroute to 10.1.1.2 (10.1.1.2), 30 hops max, 60 byte packets
 1  * * *
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
mininet>
```

 # 4

```
mininet> iperfudp bw=20 laptop Fridge
*** Iperf: testing UDP bandwidth between laptop and Fridge
*** Results: ['bw=20', '14.7 Kbits/sec', '14.7 Kbits/sec']
```